南工问天26赛季FreeRTOS培训课件V1.1

V1.1 author：rediduck ； CMSIS-RTOS Version：2.1.3

感谢黄梓源学长对于此课件的贡献，本课件在CMSIS-RTOS2 函数介绍大量借鉴了黄梓源学长的学习笔记

本课件致力于帮助大家浅显的理解什么是 FreeRTOS，如何初步使用 FreeRTOS，深入层面还需大家自行探索

Chapter 1 线程

1、线程与进程的概念

- **进程**：操作系统结构的基础，可以分配给处理器并由处理器执行的一个实体，包含程序的代码、运行状态以及资源（任务管理器）
- **线程**：操作系统能够进行运算调度的最小单元，被包含在进程中，是进程的实际运作单位
- **总结**： 一个操作系统可以同时执行多个进程

  一个进程可以执行多个线程

2、STM32 上的线程

不使用 RTOS（裸机开发）

不存在线程的概念，存在多个执行流，如：主执行流、中断服务函数

```c
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_CAN1_Init();
  MX_CAN2_Init();
  MX_TIM6_Init();
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Init scheduler */
  osKernelInitialize();  /* Call init function for freertos objects (in cmsis_os2.c) */
  MX_FREERTOS_Init();

  /* Start scheduler */
  osKernelStart();

  /* We should never get here as control is now taken by the scheduler */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

你的程序就会单一的在 main.c 文件中的 main 函数的 while 循环中单一执行

好处:

- 简单易写，按顺序执行的代码可以抚平草履虫般的头脑

- 比RTOS更快、确定性更强

坏处：容易写成屎山，不宜维护

无法实现复杂功能（可能需要你在 while 循环中搓一个类似于任务调度器的东西）

使用FREERTOS

线程被称为任务，是 FREERTOS 调度的基本单位，可以把每个任务理解为使用裸机开发时的一个 while 循环理论上来说，

只要栈空间足够，就可以创建很多个任务（不要很多），不必再把不同机构的代码都放在同一个循环中

好处：

- 结构更清晰
- 更容易写复杂系统
- 更容易维护

坏处：需要考虑不同任务之间的优先级关系，需要考虑任务的空间使用以及使用同一资源任务的冲突

总结

- 对于校内赛那样控制功能简单，任务单一的车，用裸机开发十分方便迅速
- 对于正赛这样复杂的、多人维护的项目，一个STM32需要控制多个机构且多个机构可能同时执行自己的控制逻辑，主控板**严禁**使用裸机开发

Chapter 2 FREERTOS

1、使用原因

Free Real-Time Operating System：顾名思义，免费实时操作系统

对于战队机器人的开发来说，我们非常在意代码运行的实时性

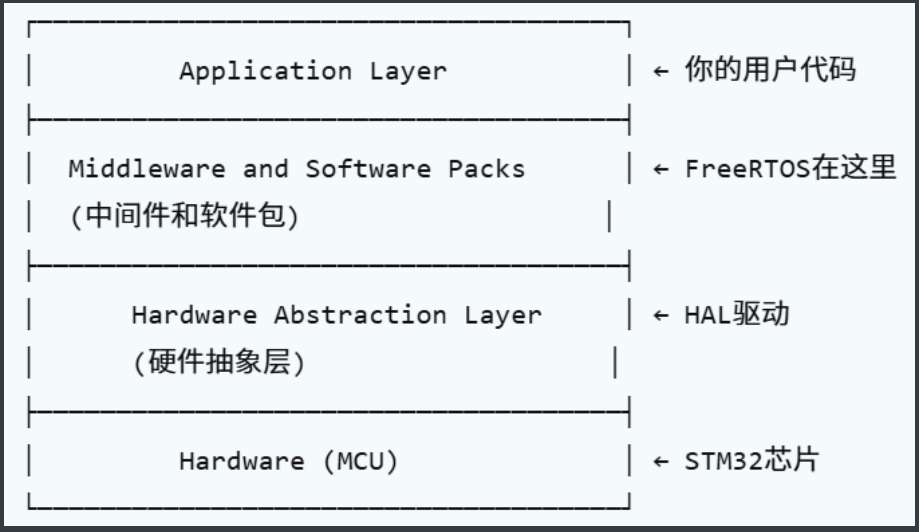
> 例如：遥控器通过串口传来消息，要去实时的解析，否则会变成高pin战士，操作手会发出尖锐的爆鸣

> 通过PID控制电机时，我们不希望PID计算的会有很高延迟，否则电机在开环运行下可能会跑飞
>
> 我们希望通过状态机实时掌握各个机构、各个电机的状态，从而可以将机器的控制牢牢掌握在手里

- 用一个 while 循环+中断来控制复杂的机构会很难维护，最好要多个 while +硬中断+软中断相互合作来实现功能
- 对于各个初学者来说，FREERTOS 的学习相比于学习用通用操作系统加实时补丁，学习成本更低，便于直接上手
- STM32 稳定性好，CubMX 对 FREERTOS 的支持良好，我们不必繁琐地去手动移植

综上所述，**FREERTOS** 能很好的满足多任务并发、生态支持良好、学习成本低、资金吃紧等等要求，所以选择 FREERTOS

2、基本概念



1、FREERTOS 构建了用户代码与硬件之间的中间层，通过调用 FREERTOS 的函数创建任务（任务可以简要的理解为一个 while 循环），FREERTOS 的内核调度器 可以让你的多个while循环看起来像是在同时执行，就像是在**用很多个单片机控制同一个机器**一样，这多个单片机共用数据、串口等等。

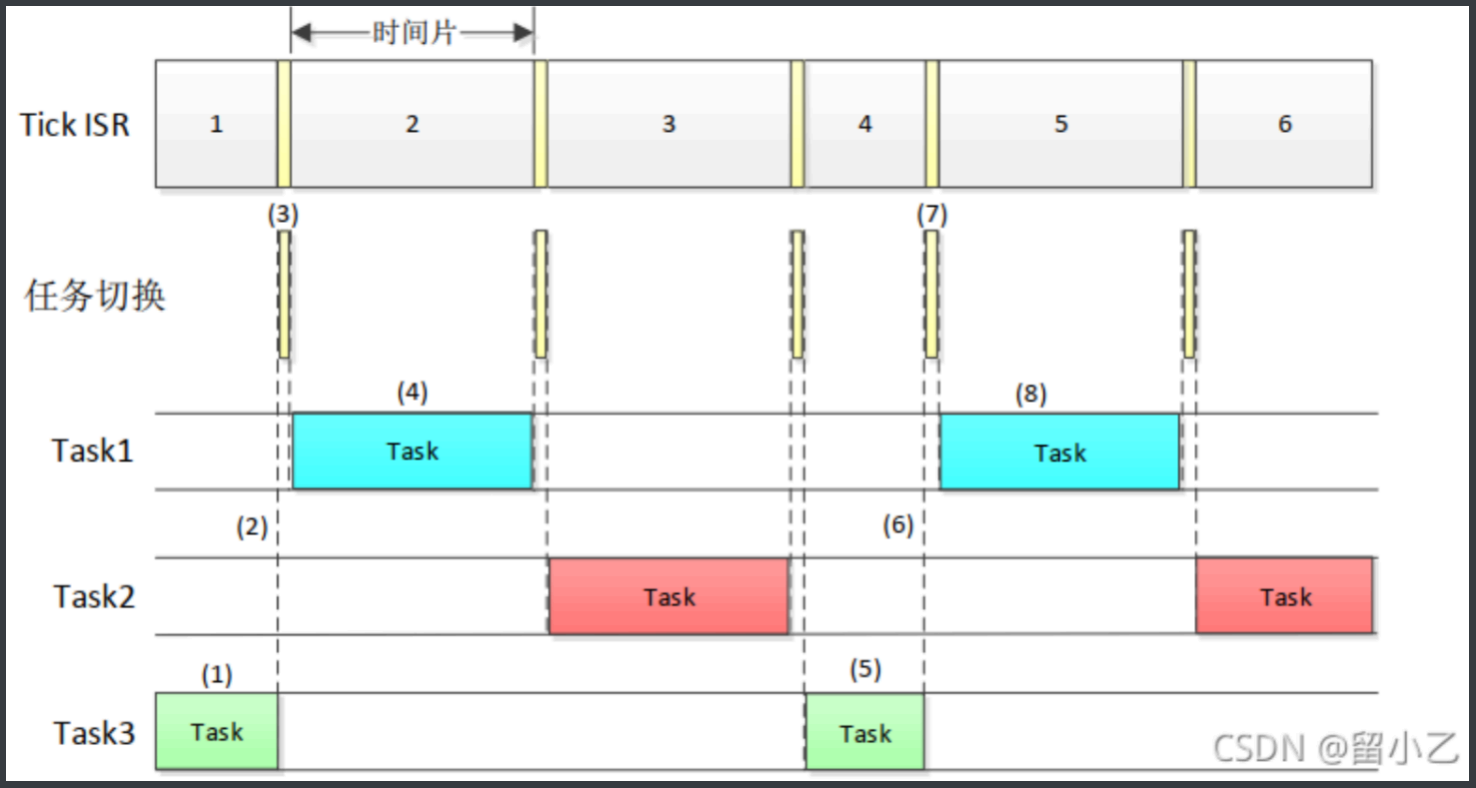
2、FREERTOS 只是让不同任务在宏观层面上（使用）看起来像是不同任务在同时进行，实际上因为只有一个MCU，它**只能一行一行地执行代码**，FRREERTOS通过内核调度器让不同任务在一个大的时间上（比如1s）瓜分时间片（1ms）（大致上看来如此，实际还有抢占更复杂的机制），从而让**宏观层面**上看来在这个大的时间内不同地任务都在执行。

3、并不是说给一个任务分一个时间片，那个任务执行完后MCU就傻等着什么都不干，FREERTOS的任务调度器切换MCU执行不同任务是多重机制的，是**定时检查+事件驱动**类型的。

> 例如：每个时间片会检查一次哪些任务就绪了，哪些任务需要切换了等等。
>
> 在特定函数执行时主动进行任务切换，如 xQueueSend()、xTaskNotifyGive()、taskYIELD()、vTaskDelay()（内部调用 taskYIELD()），因为一般一个　任务执行一遍while中的代码在μs级别，而时间片的量级在1ms，所以如果不是在任务的while循环中再加while循环，一般一个任务用不满一个时间　片。

4、任务一共有**运行、就绪、阻塞、挂起、删除**五种状态

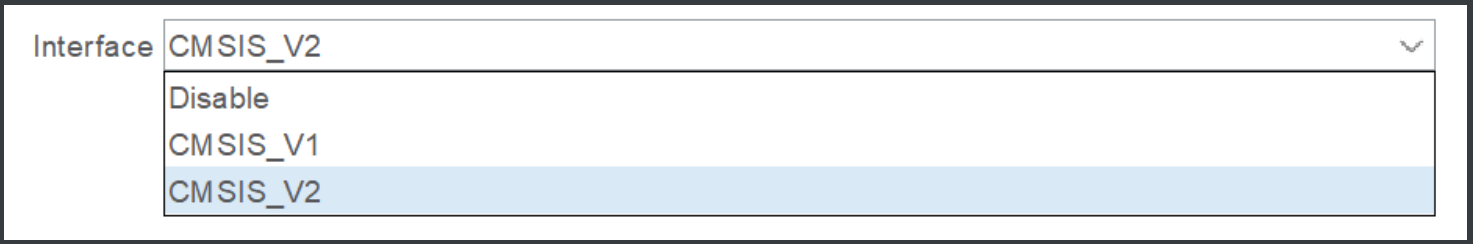
- 就绪态：任务目前的状态可以被执行，只是MCU正在执行其他任务。
- 运行态：在一个时刻只可能有一个任务处于运行态，若有优先级更高的任务加入了就绪态，则目前运行的任务会立刻停止运行变为就绪态。相同优先级的任务会在调度器的分配下分配时间片依次运行。如果高优先级的任务一直运行，那么处于就绪态的低优先级的任务就会一直得不到运行。
- 挂起态：代表任务被冻结了，不参与调度器的调度。除非人为唤醒。
- 阻塞态：表示任务正在等待某个事件或者时间条件，在条件满足之后任务会被加入就绪态。
- 删除态：任务已经终止，资源正在等待或已经被清理。



Chapter3 CMSIS-RTOS2（重点）

1、基础概念

CMSIS-RTOS2 是 ARM 为 Cortex-M 微控制器设计的**第二代实时操作系统接口标准**。它是 CMSIS（Cortex Microcontroller Software Interface Standard）生态系统的一部分，提供了一套标准化的 RTOS API，使应用程序能够在不同的 RTOS 实现之间无缝移植。



CubMX为我们准备好了使用 FREERTOS 的API，我们无需手动移植、甚至无需关注使用的是什么RTOS，**唯一**需要做的就是详细了解CMSIS-RTOS2为我们准备好的各个函数，根据自己的需求合理使用相应函数就可以实现多线程的开发了。

总而言之就是使用了CMSIS-RTOS2准备好的函数，只要能在不同的MCU上移植成功，就可以使同一套代码在不同MCU上、不同的RTOS上成功的跑起来，大大降低了开发的负担。总而言之就是 CubMX 已经为我们准备好了所有的前置条件，现在要做的就是：**1、学习 CMSIS-RTOS2 API 2、会在CubMX上配置 FREERTOS**

2、CMSIS-RTOS2 API接口介绍

这部分并不会全部介绍，会挑一些重点进行介绍，详情还得自行前往CMSIS-RTOS2官网学习

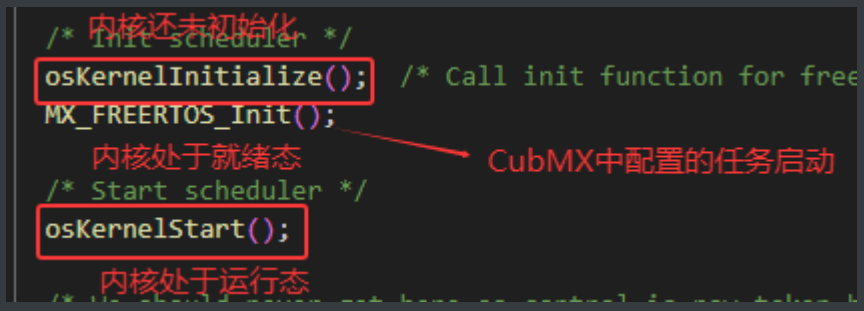
内核信息与控制

数据结构

- `struct osVersion_t` 标识底层的 RTOS 内核和 API 版本号

- ```
  enum osKernelState_t
  {
      osKernelInactive = 0,           //内核未初始化
      osKernelReady = 1,              //内核已初始化，准备好运行
      osKernelRunning = 2,            //内核正在运行
      osKernelLocked = 3,             //内核锁定
      osKernelSuspended = 4,          //内核挂起
      osKernelError = -1,             //发生错误
      osKernelReserved = 0x7FFFFFFFU  //保留，无意义
  }
  ```

  绝大多数情况下（只要我们不去手动操作内核），内核就会保持在正在运行的状态



CubMX在生成代码的时候就已经帮我们在main.c中调用了内核的初始化函数和内核的启动函数，编写代码时无需关注内核启动

函数

- `osStatus_t osKernelInitialize(void)`

  用于初始化内核。**不能在中断中使用**

  返回值：

  - `osOK` 初始化成功
  - `osError` 发生错误
  - `osError` 从中断中使用了该函数
  - `osErrorNoMemory` 没有足够的内存用于此操作
- `osKernelState_t osKernelGetState(void)`

  用于获取内核当前状态，**要在内核初始化后才能调用！可以在中断中使用**

  返回值：内核状态
  ```

- `osStatus_t osKernelStart(void)`

  用于启动内核和开始任务调度，如果启动成功就不会有返回值！换句话说就是调用的个函数的文件中下文就不会被执行，控制权已经给到了调度器

  不能在中断中使用

```
/* Start scheduler */
osKernelStart();

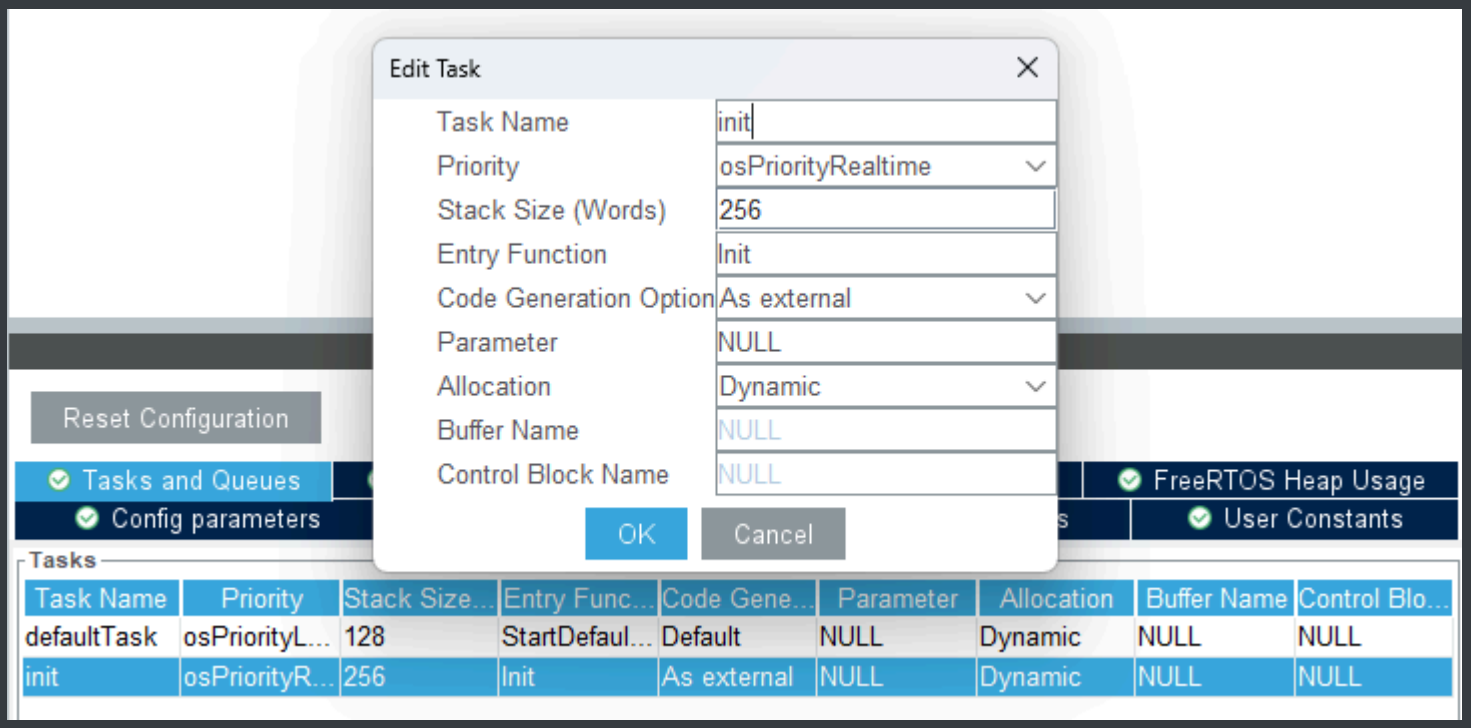
/* We should never get here as control is now taken by the scheduler */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
```

注意：CubMX会帮我们生成一个默认启动任务，这个任务无法在CubMX上删除也无法用外部定义，目前采取的做法是自己建一个启动任务选择As external,

这样就可以愉快的在UserCode中写初始任务而不用去手动修改CubMX为我们写好的文件。



- `int32_t osKernelLock(void)`

  用于锁定调度器，停止所有任务**切换**，硬中断依然会执行。**不能在中断中使用**

返回值：

- 或是调用该函数前内核的状态（1：锁定；0：未锁定）
- 或是 `osError` 发生错误； `osErrorISR` 从中断服务函数中调用

- `int32_t osKernelUnlock(void)`

  用于解锁内核，**不能在中断中使用**

  返回值：

  - 或是调用该函数前内核的状态（1：锁定，0：未锁定）
  - 或是 `osError` 发生错误； `osErrorISR` 从中断服务函数中调用

任务管理

数据结构

- `struct osThreadAttr_t` 用于配置任务的属性

```
typedef struct {
      const char                  *name;    ///< 任务名称（常用）
      uint32_t                  attr_bits;  ///< 任务属性位掩码（不常用），默认以分离模式创建，任务在终止时会自动释放资源
      void                       *cb_mem;   ///< 指向任务控制块内存的指针（不常用），默认动态分配内存
      uint32_t                    cb_size;  ///< 提供TCB内存大小（字节）（不常用），默认为0表示未提供TCB内存
      void                     *stack_mem;  ///< 不指向任务内存的指针，默认从内存池动态分配栈
      uint32_t                 stack_size;  ///< 任务栈的大小（常用），如果任务的内存设置过小会产生内存溢出的问题导致程序不能正常运行
      osPriority_t              priority;   ///< 任务的优先级（常用），高优先级的任务处于就绪态时可以打断低优先级的任务
      TZ_ModuleId_t             tz_module;  ///< 安全拓展模块标识符（不常用），默认不启用
      uint32_t                  reserved;   ///< 保留字段（不用管），默认为0
      } osThreadAttr_t;
```

```
/* ===== 抽屉控制任务 =======*/
osThreadId_t          drawerUPHandle;
const osThreadAttr_t drawerUP_attributes = {
    .name       = "drawerUP",
    .stack_size = 128 * 8,
    .priority   = (osPriority_t) osPriorityHigh,
};
```

这是正赛代码中的一段配置，用来创建抽屉机构的任务的属性，一般创建属性的时候只需要关注图中三个变量即可，其他不赋值即是默认值

- `osThreadId_t` 任务句柄类型，相当于每个任务的身份证
- `enum osThreadState_t` 任务的状态

```
    enum osThreadState_t
    {
        osThreadInactive = 0,           //该任务已创建但未被主动使用，或已被终止,可能任务被挂起了
        osThreadReady = 1,              //任务已准备好执行，但目前没有运行
        osThreadRunning = 2,            //任务正在运行
        osThreadBlocked = 3,            //该任务当前被阻止（延迟，等待事件或暂停）
        osThreadTerminated = 4,         //该任务被终止并释放其所有资源
        osThreadError = -1,             //任务引发了错误情况，无法进行调度
        osThreadReserved = 0x7FFFFFFF   //保留，无意义
    }
```

- `enum osPriority` 任务的优先等级，是RTOS调度器层面的概念

```
    enum    osPriority_t {
      osPriorityNone = 0,
      osPriorityIdle = 1,
      osPriorityLow = 8,
      osPriorityLow1 = 8+1,
      osPriorityLow2 = 8+2,
      osPriorityLow3 = 8+3,
      osPriorityLow4 = 8+4,
      osPriorityLow5 = 8+5,
      osPriorityLow6 = 8+6,
      osPriorityLow7 = 8+7,
      osPriorityBelowNormal = 16,
```

```
    osPriorityBelowNormal1 = 16+1,
    osPriorityBelowNormal2 = 16+2,
    osPriorityBelowNormal3 = 16+3,
    osPriorityBelowNormal4 = 16+4,
    osPriorityBelowNormal5 = 16+5,
    osPriorityBelowNormal6 = 16+6,
    osPriorityBelowNormal7 = 16+7,
    osPriorityNormal = 24,
    osPriorityNormal1 = 24+1,
    osPriorityNormal2 = 24+2,
    osPriorityNormal3 = 24+3,
    osPriorityNormal4 = 24+4,
    osPriorityNormal5 = 24+5,
    osPriorityNormal6 = 24+6,
    osPriorityNormal7 = 24+7,
    osPriorityAboveNormal = 32,
    osPriorityAboveNormal1 = 32+1,
    osPriorityAboveNormal2 = 32+2,
    osPriorityAboveNormal3 = 32+3,
    osPriorityAboveNormal4 = 32+4,
    osPriorityAboveNormal5 = 32+5,
    osPriorityAboveNormal6 = 32+6,
    osPriorityAboveNormal7 = 32+7,
    osPriorityHigh = 40,
    osPriorityHigh1 = 40+1,
    osPriorityHigh2 = 40+2,
    osPriorityHigh3 = 40+3,
    osPriorityHigh4 = 40+4,
    osPriorityHigh5 = 40+5,
    osPriorityHigh6 = 40+6,
    osPriorityHigh7 = 40+7,
    osPriorityRealtime = 48,
    osPriorityRealtime1 = 48+1,
    osPriorityRealtime2 = 48+2,
    osPriorityRealtime3 = 48+3,
    osPriorityRealtime4 = 48+4,
    osPriorityRealtime5 = 48+5,
    osPriorityRealtime6 = 48+6,
```

```
    osPriorityRealtime7 = 48+7,
    osPriorityISR = 56,
    osPriorityError = -1,
    osPriorityReserved = 0x7FFFFFFF
  }
```

与硬件中断相反，软件层面的任务任务优先级数字**越大**，优先级**越高**

不过要注意的一点是，CubMX中配置的NVIC中断的优先级和这里的任务优先级有本质区别。

要认识到：

- 每个任务的死循环相当于不用FreeRTOS时main函数的while循环，任务的优先级是给任务调度器看的，不影响中断行为。任务的优先级再高，当中断发生时，也会去执行中断回调函数。
- FreeRTOS任务调度器切换不同任务靠的是**SysTick中断**和**PendSV中断**，也就是前文所说的多重任务切换（定时切换+事件切换）。

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority	Uses FreeRTOS functions
Non maskable interrupt	☑	0	0	☐
Hard fault interrupt	☑	0	0	☐
Memory management fault	☑	0	0	☐
Pre-fetch fault, memory access fault	☑	0	0	☐
Undefined instruction or illegal state	☑	0	0	☐
System service call via SWI instruction	☑	0	0	☐
Debug monitor	☑	0	0	☐
Pendable request for system service	☑	15	0	☑
System tick timer	☑	15	0	☑
PVD interrupt through EXTI line 16	☐	5	0	☑
Flash global interrupt	☐	5	0	☑
RCC global interrupt	☐	5	0	☑
CAN1 TX interrupts	☐	5	0	☑
CAN1 RX0 interrupts	☑	5	0	☑
CAN1 RX1 interrupt	☐	5	0	☑
CAN1 SCE interrupt	☐	5	0	☑
USART1 global interrupt	☑	5	0	☑
TIM6 global interrupt, DAC1 and DAC2 underrun er...	☑	5	0	☑
Time base: TIM7 global interrupt	☑	15	0	☐
DMA2 stream2 global interrupt	☑	5	0	☑
CAN2 TX interrupts	☐	5	0	☑
CAN2 RX0 interrupts	☑	5	0	☑
CAN2 RX1 interrupt	☐	5	0	☑
CAN2 SCE interrupt	☐	5	0	☑
FPU global interrupt	☐	5	0	☑

可以看到**SysTick中断、PendSV中断**和负责HAL库时基维护的**TIM7定时器中断**的优先级是非常低的，而CAN，DMA等中断优先级都高于它们。

函数

- `void (*osThreadFunc_t)(void* argument)` 任务函数原型（告诉我们任务函数应该怎么写）
- `osThreadId_t osThreadNew(osThreadFunc_t func, void* argument, const osThreadAttr_t* attr)`

  用于创建任务。**不能在中断中使用**

  传入参数：

  - `func`：任务函数的函数指针，一般情况下传入函数名就可以了，编译器会帮你自动转化为函数指针
  - `argument`：传递给任务的参数，也就是写函数时参数的实例化
  - `attr`：任务属性，传入NULL为默认

  返回值：

  - 任务的独特ID，可以用这个ID通过其他函数来获取这个任务的状态

```
osThreadId_t      drawerOUTHandle;  // 任务的句柄（ID）

const osThreadAttr_t drawerOUT_attributes = {

 .name     = "drawerOUT",

 .stack_size = 128 * 8,

 .priority   = (osPriority_t) osPriorityHigh,

}; // 任务属性

void DrawerOUT_Control(void* argument)
{
 for (;;)
 {

    //具体的任务执行逻辑

 }
 osDelay(10);
}//任务函数
```

```
drawerOUTHandle = osThreadNew(DrawerOUT_Control, NULL,
&drawerOUT_attributes);
```

- `osThreadState_t osThreadGetState(osThreadId_t threa_id)`

  用于获取目标任务的状态，**不能在中断中使用**

  传入参数：

  - `thraead_id`：任务ID

  返回值：

  - 任务状态，详情见任务状态

- `osStatus_t osThreadYield(void)`

  在任务中调用，用于立刻切换到下一个相同优先级且处于就绪态的任务，如果没有相同优先级的任务处于就绪态，那么当前任务就会继续执行。

  不能在中断中使用

  可能返回值：

  - `osOK`：成功切换
  - `osError`：出现错误
  - `osErrorISR`：函数错误地被在中断服务函数中调用

- `osStatus_t osThreadSuspend(osThreadId_t thread_id)`

  用于挂起任务。**不能在中断中使用**

  传入参数：

  - `thraead_id`：任务ID

  可能返回：

  - `osOK`：目标任务成功挂起
  - `osErrorParameter`：传入的目标任务的句柄不存在
  - `osErrorResource`：传入的目标任务为无效状态
  - `osErrorISR`：函数被错误地从中断回调函数中调用

- `osStatus_t osThreadResume(osThreadId_t thread_id)`

  用于解挂任务。**不能在中断中使用**

  传入参数：

  - `thraead_id`：任务ID

可能返回：

- `osOK`： 目标任务成功解挂
- `osErrorParameter`： 传入的目标任务的句柄不存在
- `osErrorResource`： 传入的目标状态为无效状态
- `osErrorISR`： 函数被错误地从中断回调函数中调用

- `void osThreadExit(void)` （好用捏）

  用于退出任务函数。**不能在中断中使用**

- `osStatus_t osThreadTerminate(osThreadId_t thread_id)`

  用于删除目标任务。**不能在中断中使用**

  注意：如果任务用的是可连接模式（FreeRTOS原生并没有这个模式，CMSIS-RTOS2自己拓展实现了），调用这个函数后内存并没有被释放，可能会发生内存溢出问题

  tips：一般我们在创建任务时都是用的默认的分离模式，对于这种模式的任务来说，在函数本身用osThreadExit()和用osThreadTerminate(ID)是没什么区别的，注意如果任务没有写成死循环，不要用return或者直接不写来退出，对于CubMX移植的CMSIS-RTOS2会存在严重问题。

  传入参数：

  - `thraead_id`： 任务ID

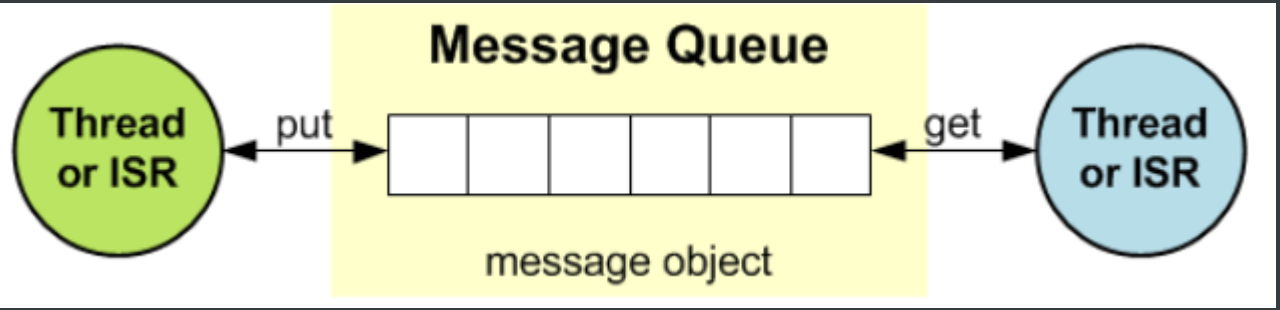
  可能返回：

  - `osOK`： 目标任务成功解挂
  - `osErrorParameter`： 传入的目标任务的句柄不存在
  - `osErrorResource`： 传入的目标状态为无效状态或者目前没有其他可以就绪的任务（因为调度器至少要有一个可执行的任务）
  - `osErrorISR`： 函数被错误地从中断回调函数中调用

消息队列

用处

在任务之间交换信息，类似于FIFO（一个任务向队列放入信息，另一个任务从中拿出信息）



数据结构

- `struct osMessageQueueAttr_t`

```
typedef struct {
  const char                  *name;   ///< 消息队列名称
  uint32_t                   attr_bits;   ///< 属性位，当前必须设置为0
  void                        *cb_mem;    ///< 控制块内存指针，控制块：存储消息队列内部状态的数据结构
  uint32_t                    cb_size;   ///< 控制块内存大小（一般用默认的动态分配就可以了）
  void                        *mq_mem;    ///< 消息数据存储区指针
  uint32_t                    mq_size;   ///< 消息数据存储区大小
} osMessageQueueAttr_t;
```

函数

- `osMessageQueueId_t osMessageQueueNew(uint32_t msg_count, uint32_t msg_size, const osMessageQueueAttr_t* attr)`

  创建一个队列。**不能在中断中使用**

  传入参数：

  - `msg_count`：队列深度
  - `msg_size`：一个消息的大小（单位：字节）
  - `attr`：队列的属性，若传入NULL为默认

  返回值：

  - 队列句柄

- `osStatus_t osMessageQueuePut(osMessageQueueId_t mq_id, const void* msg_ptr, uint8_t msg_prio, uint32_t timeout)`

  用于队列中写入信息（注意是**写入**，是值传递）。**如果timeout设置为0则可以在中断中使用**

传入参数：

- `mq_id`：队列句柄
- `msg_ptr`：写入的消息指针，注意要和队列每个消息大小相匹配
- `msg_prio`：消息优先级（数字越大，优先级越大）
- `timeout`：最大消息等待时间（当队列已满时，当前任务应该等待多久）
  - 0：函数立刻返回（消息丢弃）
  - osWaitForever：该函数将无限期地等待消息发送
  - 0~osWaitForever：等待的时间（单位：内核滴答数）

返回参数：

- `osOK`：消息已经放入队列
- `osErrorTimeout`：给定时间内无法将消息放入队列
- `osErrorResource`：当 `timeout` 为0时队列已满
- `osErrorParameter`：队列句柄不存在

- `osStatus_t osMessageQueueGet(osMessageQueueId_t mq_id, const void* msg_ptr, uint8_t* msg_prio, uint32_t timeout)`

用于从队列中读取信息。**如果timeout设置为0则可以在中断中使用**

传入参数：

- `mq_id`：队列句柄
- `mag_ptr`：读取信息后存入的内存指针
- `msg_prio`：读取消息的优先级后存入的内存指针（一般用不到，传入NULL即可）
- `timeout`：调用一次这个函数时的最大等待时间
  - 0：函数立刻返回（不管读没读到数据）
  - osWaitForever：该函数将无限期地等待队列中传来消息
  - 0~osWaitForever：等待的时间（单位：内核滴答数）

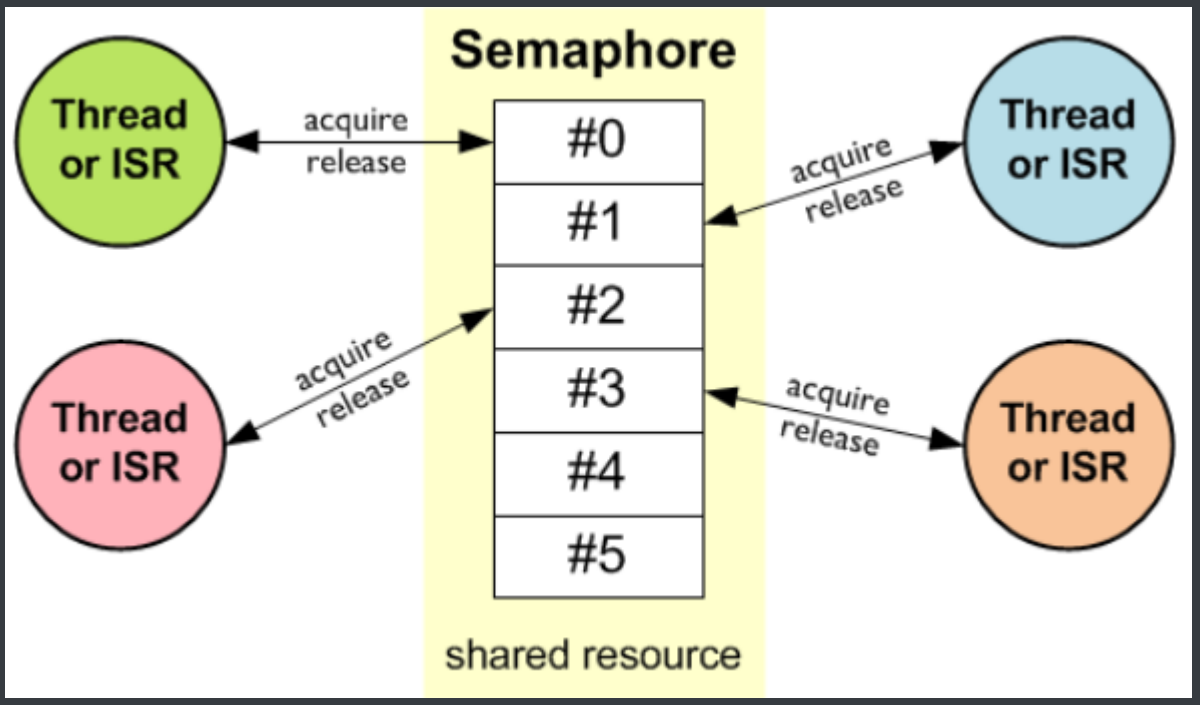
返回参数：

- `osOK`：消息已经从队列中接收
- `osErrorTimeout`：给定时间内队列中没有消息
- `osErrorResource`：当 `timeout` 为0时调用该函数时队列没有消息
- `osErrorParameter`：队列句柄不存在

信号量

用处

信号量用于管理和保护对资源的访问，例如使得不超过一定数量的任务或中断服务函数访问一组共享资源。



本质上是一个高级一点的计数器。

数据结构

- `struct osSemaphoreAttr_t`

```
typedef struct {
  const char                  *name;   ///< 信号量的名称
  uint32_t                 attr_bits;   ///< 保留位（没用），置0
  void                      *cb_mem;   ///< 控制块内存指针
  uint32_t                   cb_size;   ///< 控制块内存大小（单位：字节）
} osSemaphoreAttr_t;
```

函数

- `osSemaphoreId_t osSemaphoreNew(uint32_t max_count, uint32_t initial_count, const osSemaphoreAttr_t* attr)`

  用于**创建**信号量。**不能在中断中使用**

注意不可以在内核初始化前创建

传入参数：

- `max_count`：可用令牌最大数量
- `initial_count`：可用令牌的初始数量
- `attr`：信号量属性，传入NULL为默认

返回值：

- 成功：信号量句柄
- 失败：NULL

- `osStatus_t osSemaphoreAcquire(osSemaphoreId_t semaphore_id, uint32_t timeout)`

用于***获取***信号量，获取信号量后处理完任务记得要手动的**给出**信号量。**如果timeout设置为0则可以在中断中使用**

传入参数：

- `semaphore`：信号量句柄
- `timeout`：阻塞超时时间（单位：内核滴答）
  - 0：调用该函数时不管信号量是否还有剩余，都会立刻返回
  - osWaitForever：调用该函数时在信号量可获取之前会阻塞在这个函数，会把调用这个函数的任务置为阻塞态
  - 0~osWaitForever：如果信号量可获取，则会立刻返回；否则会阻塞指定的时间再返回

返回值：

- `osOK`：成功获取信号量
- `osErrorTimeout`：在给定的时间内未能获取信号量
- `osErrorResource`：当timeout为0时，调用该函数时没有获取到信号量
- `osErrorParameter`：传入的句柄错误

- `osStatus_t osSemaphoreRelease(osSemaphoreId_t semaphore_id)`

用于***给出***信号量。**可以在中断中使用**

传入参数：

- `semaphore_id`：信号量句柄

返回值：

- `osOK`：信号量成功给出
- `osErrorResource`：令牌无法给出，因为信号量计数已经满了（说明可能其他地方拿了一次

放了多次）

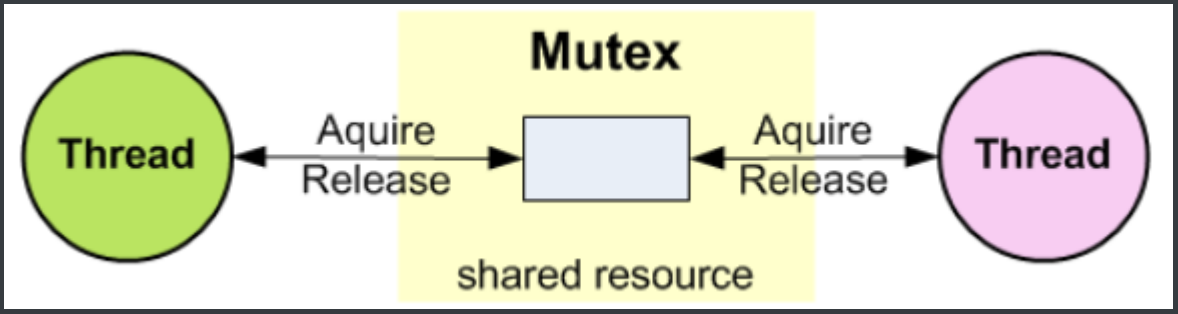
- `osErrorParameter`：信号量句柄错误

互斥量

用处

可以看作是深度为1的信号量，可以用来限制一个资源在一段时间内只被一个任务使用

但是信号量任何任务都可释放，互斥量只有持有者可以释放



数据结构

- `struct osMutexAttr_t` 用于配置互斥量的属性

```
typedef struct {
  const char                  *name;   ///< 互斥量的名字
  uint32_t                   attr_bits;   ///< 互斥量属性
  void                       *cb_mem;    ///< 控制块内存地址
  uint32_t                    cb_size;    ///< 控制块大小（单位：字节）
} osMutexAttr_t;
```

`attr_bit` 的选择

- `osMutexRecursive`：递归互斥量，允许同一任务多次获取互斥量，但是获取与释放要匹配，如果获取N次就要相应释放N次。
- `osMutexPrioInherit`：带优先级继承的互斥量，可以防止优先级反转，当高优先级任务等待同一锁时，会临时提高这个持有锁的低优先级任务，从而保证高优先级任务能及时执行。
- `osMutexRobust`：健壮互斥量，普通互斥量当持有锁的任务意外终止（崩溃、被杀死）时，普通互斥量会永远保持锁定的状态，导致其他需要持有该锁访问资源的任务就会永久等待（死锁），持有健壮互斥量的任务异常终止时，互斥量会进入不一致状态，下一个尝试获取锁的任务会收到错误码，可以选择”恢复“或”清理“互斥量。

- `0` 或默认：不具有以上属性，普通互斥量。

若想同时使用多个属性时，只需在属性间加|即可，如 `osMutexRecursive|osMutexPrioInherit`

函数

- `osMutixId_t osMutexNew(const osMutesAttr_t* attr)`

  用于**创建**互斥量。**不能在中断中使用**

  传入参数：互斥量属性

  返回值：

  - 成功创建互斥量：互斥量句柄
  - 创建失败：NULL
- `osStatus_t osMutexAcquire(osMutexId_t mutex_id, uint32_t timeout)`

  用于**获取**互斥量。**不能在中断中使用**

  传入参数：

  - `mutex_id`：互斥量句柄
  - `timeout`：等待时间
    - 0：该函数立刻返回
    - osWaitForever：一直等待直到互斥量变得可获取，期间该任务会处于阻塞态
    - 0~osWaitForever：阻塞一定的时间，期间若互斥量可获取则直接返回

  返回值：

  - `osOK`：成功获取了这个互斥量，获取成功后任务会由阻塞变为就绪态
  - `osErrorTimeout`：在给定时间没有获取到锁
  - `osErrorResource`：当timeout为0时，调用该函数时没有获取到锁
  - `osErrorParameter`：互斥锁句柄错误
  - `osErrorISR`：错误地从中断服务程序中调用了该函数
- `osStatus_t osMutexRelease(osMutexId_t mutex_id)`

  用于**给出**互斥量。**不能在中断中使用**

  传入参数：互斥锁句柄

  返回值：

  - `osOK`：成功释放

- `osErrorResource`：互斥锁没有成功的拿到过或者目前拿锁的任务不是当前释放锁的任务
- `osErrorParameter`：互斥锁句柄错误
- `osErrorISR`：错误地从中断服务程序中调用该函数

事件标志

用处

类似于一种高端的标志位组，可以用来对不同任务进行**同步**，一个事件标志有32个独立的位，一个任务可以向这（32-1）位置位，然后另一个任务可以读取这个事件的32位，从而知道要做什么。

但是要注意底部实现如果是FreeRTOS的情况只有低24位可用

数据结构

- `struct osEventFlagsId_t` 用于配置事件标志的属性

```
typedef struct {
  const char                  *name;   ///< 事件标志名称
  uint32_t                 attr_bits;   ///< 保留位，必须置0
  void                      *cb_mem;    ///< 控制块内存地址，默认或NULL为动态分配
  uint32_t                   cb_size;   ///< 控制块内存大小
} osEventFlagsAttr_t;
```

函数

- `osEventFlagsId_t osEventFlagsNew(const osEventFlagsAttr_t* attr)`

  用于创建事件标志。**不能在中断中使用**

  传入参数：事件标志属性，NULL表示默认

  返回值：事件标志句柄
- `uin32_t osEventFlagsSet(osEventFlagsId_t ef_id, uint32_t flags)`

  用于设置事件标志的某些位。**可以在中断中使用**

  传入参数：

  - `ef_id`：事件标志句柄
  - `flags`：指定哪些位要设置，**注意**最高位是错误的标志位，**不要设置**

  可能返回：

    - 设置后的事件标志值
    - `osFlagsErrorUnknown`：发生未知错误
    - `osFlagsErrorResource`：目标事件标志处于不可使用状态
    - `osFlagsErrorParameter`：目标事件标志的句柄不存在或者事件标志的错误位已经被置1
- `uin32_t osEventFlagsClear(osEventFlagsId_t ef_id, uint32_t flags)`

  用于清除事件标志的某些位。**可以在中断中使用**

  传入参数：
    - `ef_id`：为目标事件标志的句柄
    - `flags`：要清除的位。

  可能返回：
    - 清除前的事件标志值。
    - `osFlagsErrorUnknown`：发生未知错误
    - `osFlagsErrorResource`：目标事件标志处于不可使用状态
    - `osFlagsErrorParameter`：目标事件标志的句柄不存在或者事件标志的最高位已经被置1
- `uint32_t osEventFlagsWait(osEventFlagsId_t ef_id, uint32_t flags, uint32_t options, uint32_t timeout)`

  用于等待获取事件标志的某些位。**如果timeout设0可以在中断中使用**

  可以将正在运行的任务阻塞。

  传入参数：
    - `ef_id`：目标事件标志句柄
    - `flags`：要用来等待获取的位
    - `options`：等待模式选项
        - `osFlagsWaitAny`：`flags`指定的任何一位有效时，函数将此位置置0并返回，否则阻塞
        - `osFlagsWaitAll`：`flags`指定的所有位置都有效时，函数将所有指定的位置置0并返回，否则阻塞
        - `osFlagsNoClear`：和前两种模式搭配使用，只是不会改变事件标志中的值

      若组合使用在模式间加`|`即可，例如：`osFlagsWaitAll|osFlagsNoClear`
    - `timeout`：等待时间
        - 0：函数立刻返回

        - osWaitForever：会一直阻塞除非满足等待完成标准
        - 0~osWaitForever：若在指定阻塞时间到之前满足条件则立刻返回，否则阻塞一定时间再返回

    返回值：

    - 清除前事件标志值
    - `osFlagsErrorUnknown`：发生未知错误
    - `osFlagsErrorTimeout`： 在预期的时间预期的标志位没有被置1
    - `osFlagsErrorResource`： timeout为0时，不满足目标位置置位要求
    - `osFlagsErrorParameter`：目标事件标志的句柄不存在或者事件标志的最高位已经被置1

若多个任务都阻塞再等待**相同**的事件标志位，则当事件触发时按照优先级高低依次唤醒（若前面等待完成的处理**不会**改变事件标志的值）

任务标志

用处

任务标志可以看作是私有化的事件标志，作用更多是为了特定的任务服务，而不像事件标志那样类似于布告板之类的。随着任务的创建而自动创建，指定任务属性的cb_size分配的空间就有任务标志的部分。

函数

- `uint32_t osThreadFlagsSet(osThreadId_t thread_id, uint32_t flags)`

  用于设置任务标志值。**可以在中断中使用**

  传入参数：

  - `thread_id`：任务句柄
  - `flags`：要设置的位，**注意最高位不要设置**

  可能返回：

  - 设置后的任务通知值。
  - `osFlagsErrorUnknown`：未知错误
  - `osFlagsErrorResource`：任务处于无效状态（任务已被终止、删除或者内存块损坏）
  - `osFlagsErrorParameter`：任务句柄无效（传入NULL或者其他一些显然错误的值）或者任务标志的最高位被置位

- `uint32_t osThreadFlagsClear(uint32_t flags)`

用于清除当前正在运行的任务的任务标志的某些位。

参数1为要清除的位。**不能在中断中使用**

传入参数：要清除的位，**注意最高位不能置1**

可能返回：

- 清除前的任务通知值。
- `osFlagsErrorUnknown`：未知错误
- `osFlagsErrorISR` ：函数错误地从中断服务程序中调用
- `osFlagsErrorParameter`：传入的要清除的位中最高位被**错误**地置1

- `uint32_t osThreadFlagsWait(uint32_t flags, uint32_t options, uint32_t timeout)`

用于阻塞当前任务，直到获取任务标志的某些位。**不能在中断中使用**

传入参数：

- `flags`：要获取的位，**注意最高位不能置1**
- `options`：等待模式选项
  - `osFlagsWaitAny`：`flags` 指定的任何一位有效时，函数将此位置置0并返回，否则阻塞
  - `osFlagsWaitAll`：`flags` 指定的所有位置都有效时，函数将所有指定的位置置0并返回，否则阻塞
  - `osFlagsNoClear`：和前两种模式搭配使用，只是不会改变事件标志中的值
- timeout：
  - 0：函数立刻返回
  - osWaitForever：会一直阻塞除非满足等待完成标准
  - 0~osWaitForever：若在指定阻塞时间到之前满足条件则立刻返回，否则阻塞一定时间再返回

返回值：

- 调用此函数前的事件标志值。
- `osFlagsErrorTimeout` 阻塞超时。
- `osFlagsErrorUnknown`：未知错误
- `osFlagsErrorResource`： timeout为0时，不满足目标位置置位要求
- `osFlagsErrorParameter`：传入参数 `flags` 的最高位被错误地置1

延时

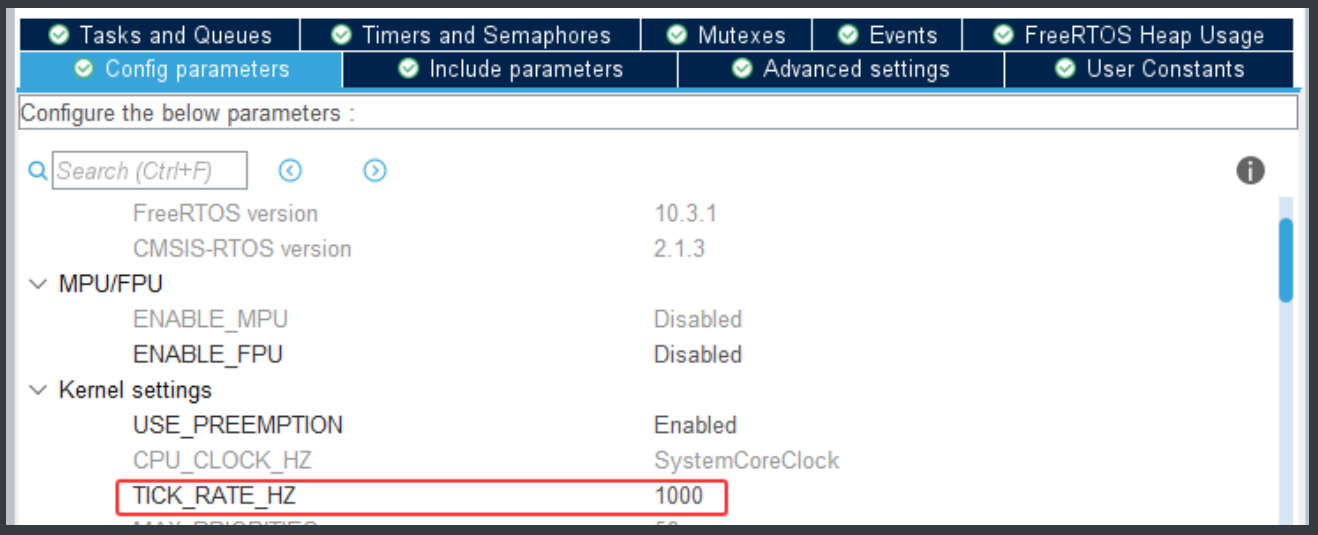
函数

- `osStatus osDelay(uint32_t ticks)`（相对延时）**不能在中断中使用**

  运行时相当于在这个函数中延长指定的时间，此时内核并不是不运行了，在一个任务的osDelay期间，这个任务是处于阻塞态的，此时MCU会去运行别的就绪态的任务，等阻塞了指定的时间会把这个任务置为就绪态。

  传入参数：

  - `ticks`：要延时的时间（单位：系统滴答数）

  返回值：

  - `osOK`：成功执行延时
  - `osErrorParameter`：传入了无效参数（0）
  - `osErrorISR`：错误地从中断服务程序中调用该函数
  - `osError`：没有成功执行延时

- `osStatus_t osDelayUntil(uint32_t ticks)`（绝对延时）**不能在中断中使用**

  当函数阻塞时逻辑和 `osDelay` 一样

  传入参数：

  - ticks：唤醒的时刻，运行到这个函数时，如果还没到这个时刻就一直阻塞，如果到了或者小于这个时刻就停止阻塞（不要超过2^{32}-1）

  返回值：

  - `osOK`：成功执行延时
  - `osErrorParameter`：传入了参数超过范围

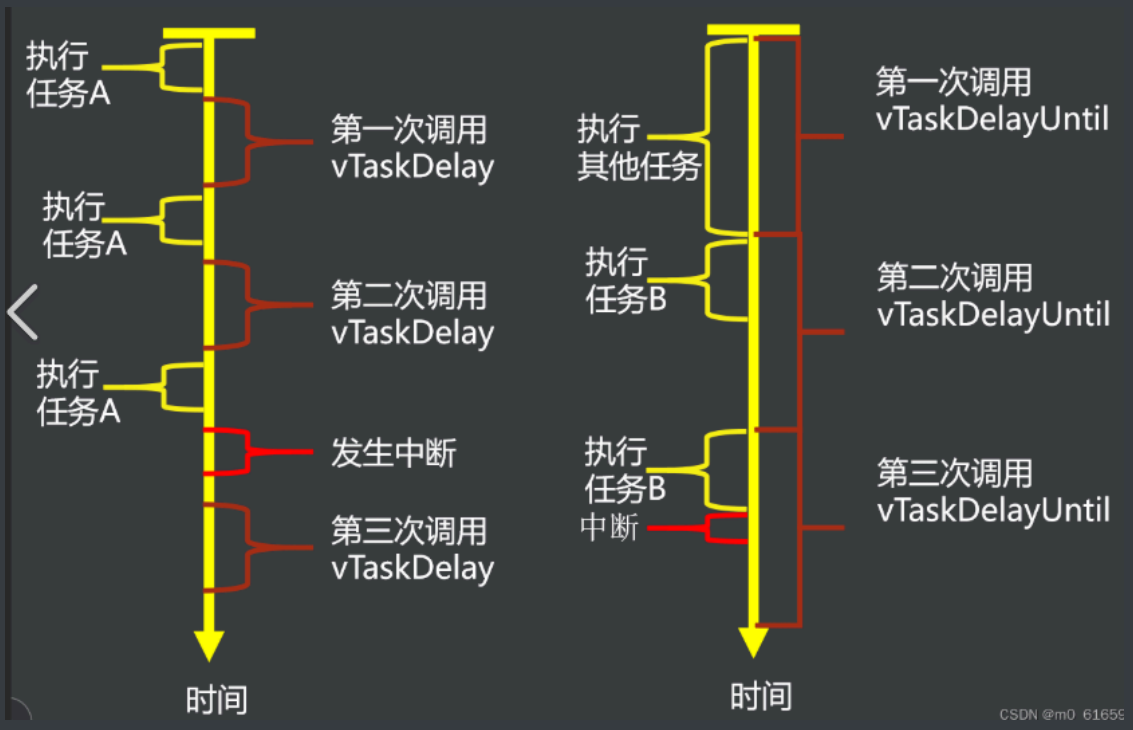
- `osErrorISR`：错误地从中断服务程序中调用该函数
- `osError`：没有成功执行延时

```
void Thread_1 (void *arg) {              // 实例任务
uint32_t tick;
tick = osKernelGetTickCount();         // 获取当前时刻
for (;;) {
tick += 1000U;                         // 设定下一次唤醒时间
osDelayUntil(tick);                    // 执行延时
// ...
}
```

函数区别

- `osDelay` 与 `osDelayUntil` 区别

```
void Task(void *arguement)
{
while(1)
{
        task    //任务函数，比如说计算赋值之类的
   delay    //延时
}
}
```



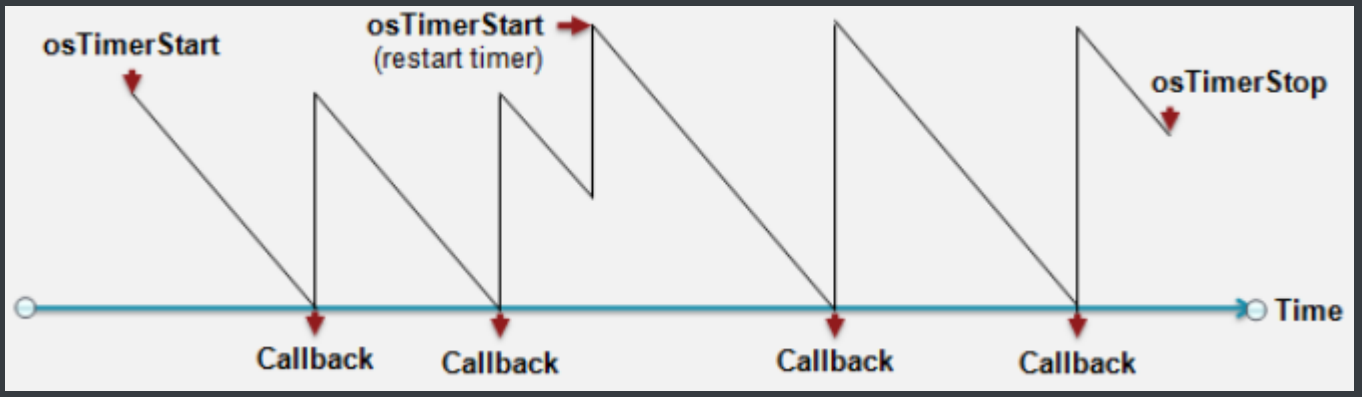
- os家族与HAL_Dealy区别

  用 `osDelay` 或者 `osDelayUntil` 时在延时的过程中是不消耗资源的，且如果调用这个函数的任务优先级比较高，在阻塞期间可以把资源释放出来给低优先级任务运行。

  用 `HAL_Delay` 时并不会把掉哟的调用该函数的任务挂入阻塞态，如果有其他同优先级的任务，就会在任务调度器分给自己的那个时间片内一直运行HAL_Delay,用完分得的时间片。如果其他就绪态任务优先级都比自己低，那么该任务就会一直运行，从而让低优先级任务饿死。

  而且当多个任务都用HAL_Delay时，可能会对延时的准确性造成影响。所以如果用CMSIS-RTOS2尽量都使用 `os` 系列，十分滴好用。

软件定时器



软件定时器并不是由硬件外设直接产生中断，而是由 **RTOS 的系统节拍中断（Tick）驱动实现的逻辑定时机制。**

RTOS 会在每一次系统 Tick 中断中更新系统时间，并检查当前是否有软件定时器到期。当某个软件定时器到期时，内核并不会在中断中直接执行回调函数，而是向一个专门的**定时器服务任务（Timer Service Task）**发送事件，由该任务在普通任务上下文中执行对应的回调函数。

一般来说F407板上TIM资源对于正赛赛说是完全足够的，因为舵机很少会用到，且用的数量不会那么大，优先推荐大家使用硬件定时器，不过软件定时器操作起来比较灵活。

数据结构

- `struct osTimerAttr_t` 用于配置定时器属性

```
typedef struct {
  const char                  *name;   ///< 定时器
  uint32_t                 attr_bits;   ///< 保留位，必须置0
  void                      *cb_mem;    ///< 控制块内存指针
  uint32_t                   cb_size;   ///< 控制块内存大小
} osTimerAttr_t;
```

函数

- `void (*osTimerFunc_t)(void* argument)`

  定时器回调函数原型，**不要放延时函数！**用CMSIS-RTOS2的API也最好用中断中可以使用的函数
- `osTimerId_t osTimerNew(osTimerFunc_t func, osTimerType_t type, void* argument, const osTimerAttr_t* attr)`

  用于创建定时器。**不能在中断中使用**

  传入参数：
  - `func`：回调函数指针，传入函数名即可
  - `type`：定时器模式选择
    - `osTimerOnce`：一次性定时器
    - `osTimerPeriodic`：周期定时器
  - `argument`：如果创建的定时器回调函数需要有外部参数传入，则在此传入
  - `attr`：定时器属性
- `osStatus_t osTimerStart(osTimerId_t timer_id, uint32_t ticks)`

  用于启动或重新启动目标定时器。**不能在中断中使用**

传入参数：

- `timer_id`：定时器句柄
- `ticks`：指定的定时器周期（单位：系统滴答）

返回值：

- `osOK`：成功启动定时器或者重新启动定时器
- `osErrorISR`：错误地从中断服务程序中调用此函数
- `osErrorParameter` 目标定时器地址不存在或周期小于等于0
- `osErrorResource` 目标定时器处于无效状态。

- `osStatus_t osTimerStop(osTimerId_t timer_id)`

用于停止目标定时器。**不能在中断中使用**

传入参数：目标定时器的句柄。

返回值：

- `osOK`：定时器成功停止
- `osErrorParameter` ：定时器句柄错误
- `osErrorResource`：定时器已经停止
- `osErrorISR`：错误地从中断服务程序中调用该函数

总结

至此所有常见常用的API接口已经介绍完了，不过在实际使用时还是会有一些细节的地方要注意

3、实际使用时一些要注意的地方

多任务系统中存在一种潜在的风险。当一个任务在使用某个资源的过程中，即还没有完全结束对资源的访问时，便被切出运行态，使得资源处于非一致，不完整的状态。如果这个时候有另一个任务或者中断来访问这个资源，则会导致数据损坏或是其它相似的错误。

例如使用电机的时候，底盘有四个电机，我们不希望用任务对四个电机获取值时当获取到了第二个电机的时候就被切出了运行态，然后下个时间片恰巧又是对这四个值进行解算的过程，此时这四个值在时间维度上就不统一，用起来可能就会出一些奇怪的问题。

所以当我们进行一些资源写入或获取时可以短暂地关闭任务调度器，让当前任务写入或获取完再切换到下一个任务，**甚至**关闭中断。

可以用互斥量进行解决，下面会介绍一种另外的解决方式

基本临界区

在CMSIS-RTOS2中没有让任务进入临界区的函数，因此我们只能借助FreeRTOS的原生函数

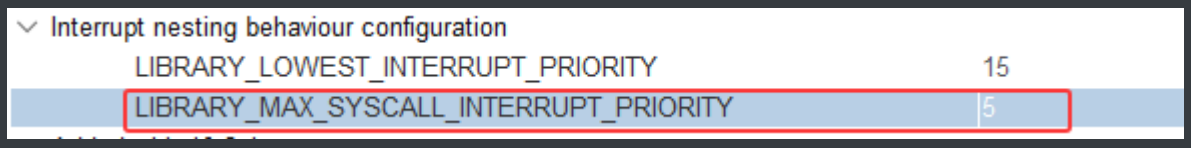
可以使用 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 指定临界区，当进入临界区后，优先级低的中断会被屏蔽，且任务调度器不会进行任务的调度。

不可以在中断服务程序中进入临界区

```
taskENTER_CRITICAL();    //临界区开始
todo;    //要做的任务
taskEXIT_CRITICAL();     //临界区结束
```

todo的东西是一口气完成的，切记里面不可以塞延时函数！一方面是为了实时性，另一方面是如果放了osDelay之类的，此时优先级为15的systick中断已经被屏蔽了，就会卡死在osDelay中，切忌切忌。此外**不要在临界区中使用CMSIS-RTOS2的API**

至于哪些中断会被屏蔽呢，CubMX中可以进行设置，所有优先级低于（指的是优先级不是数值）和等于 `LIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY` 的中断都会被屏蔽掉



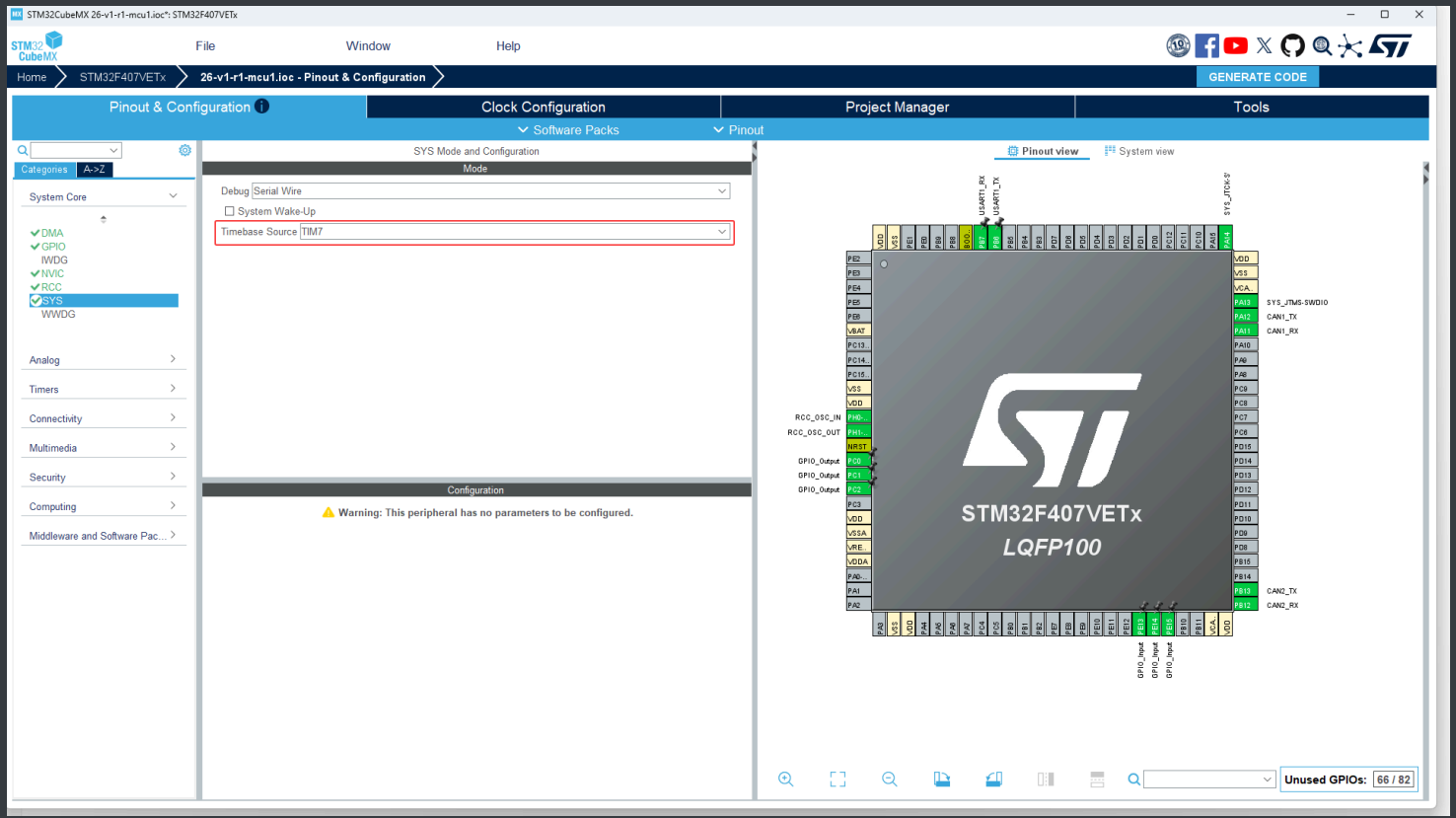
可以看到CubMX在使用FreeRTOS时会自动地把CAN、TIM、USART等中断优先级设为5，FreeRTOS可屏蔽的界限也是5

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority	Uses FreeRTOS functions
Non maskable interrupt	☑	0	0	☐
Hard fault interrupt	☑	0	0	☐
Memory management fault	☑	0	0	☐
Pre-fetch fault, memory access fault	☑	0	0	☐
Undefined instruction or illegal state	☑	0	0	☐
System service call via SWI instruction	☑	0	0	☐
Debug monitor	☑	0	0	☐
Pendable request for system service	☑	15	0	☑
System tick timer	☑	15	0	☑
PVD interrupt through EXTI line 16	☐	5	0	☑
Flash global interrupt	☐	5	0	☑
RCC global interrupt	☐	5	0	☑
CAN1 TX interrupts	☐	5	0	☑
CAN1 RX0 interrupts RCC_IRQn	☑	5	0	☑
CAN1 RX1 interrupt	☐	5	0	☑
CAN1 SCE interrupt	☐	5	0	☑
USART1 global interrupt	☑	5	0	☑
TIM6 global interrupt, DAC1 and DAC2 underrun error interrupts	☑	5	0	☑
Time base: TIM7 global interrupt	☑	15	0	☐
DMA2 stream2 global interrupt	☑	5	0	☑
CAN2 TX interrupts	☐	5	0	☑
CAN2 RX0 interrupts	☑	5	0	☑
CAN2 RX1 interrupt	☐	5	0	☑
CAN2 SCE interrupt	☐	5	0	☑
FPU global interrupt	☐	5	0	☑

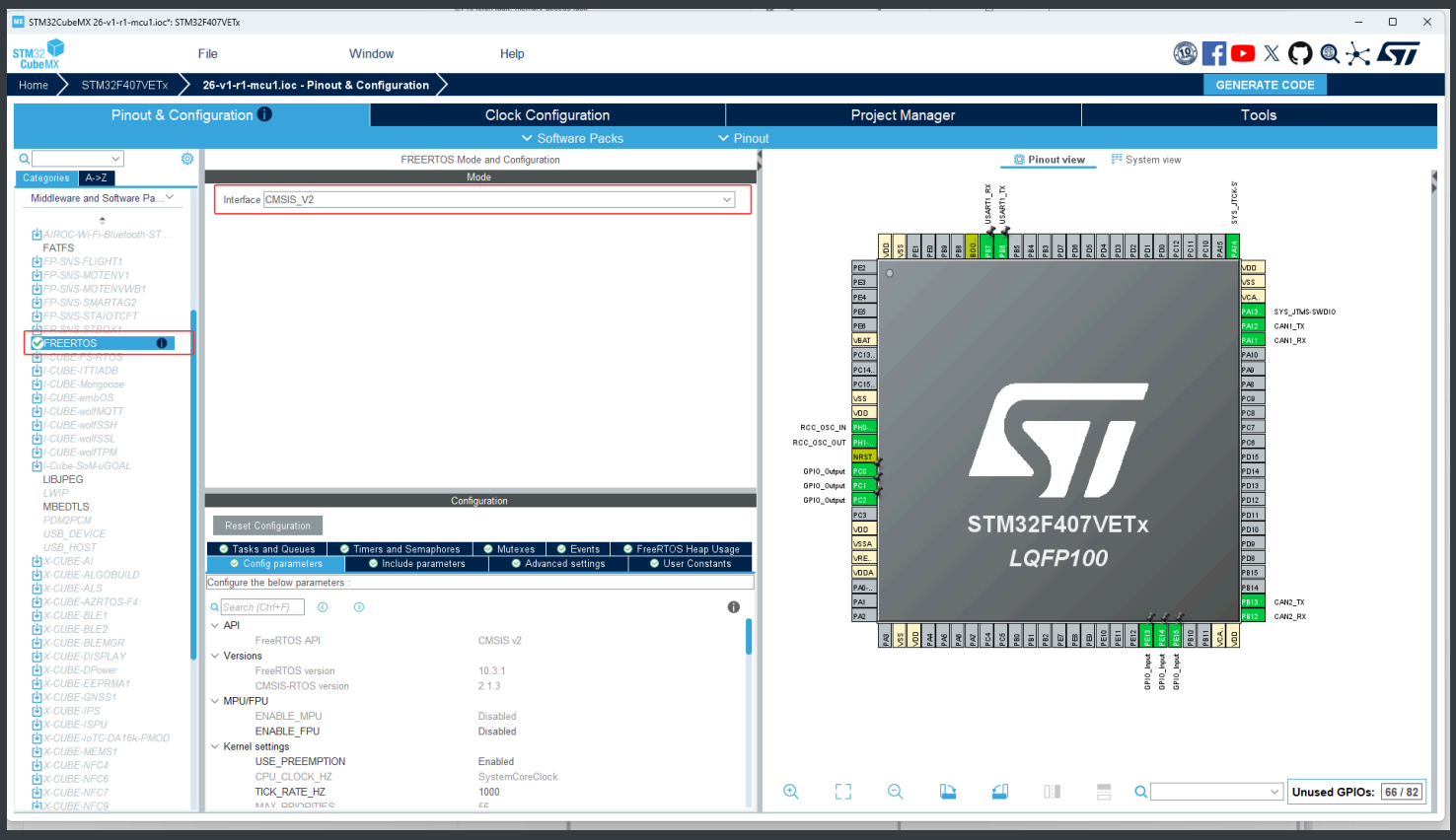
Chapter4 CubMX配置

相较于普通工程只有一点区别

1、HAL库的时基更改



2、增加FreeRTOS中间层



其他就无了，可以自行看看战队仓库借鉴我们是怎么用多线程构建工程的